TaskBot MVP — Project Scope & Architecture

Client: The Hacking Games **Date:** February 25, 2026 **Status:** Draft — for alignment before development begins

---

1. What We're Building

A serverless tool that processes Google Meet transcripts after meetings end, extracts action items using AI, and presents them to opted-in attendees for approval before creating tasks in Google Tasks.

The core loop:

1. A meeting happens on Google Meet with transcription enabled.
2. The transcript lands in Google Drive.
3. TaskBot detects it, runs AI extraction, identifies action items and owners.
4. Each opted-in attendee receives an email with their proposed tasks.
5. They click through to a lightweight web app to approve, edit, or reject.
6. Approved tasks are created in their Google Tasks with a link back to the source transcript.

---

2. Architecture Overview

Stack Decision: Firebase + TypeScript

Why Firebase over alternatives:

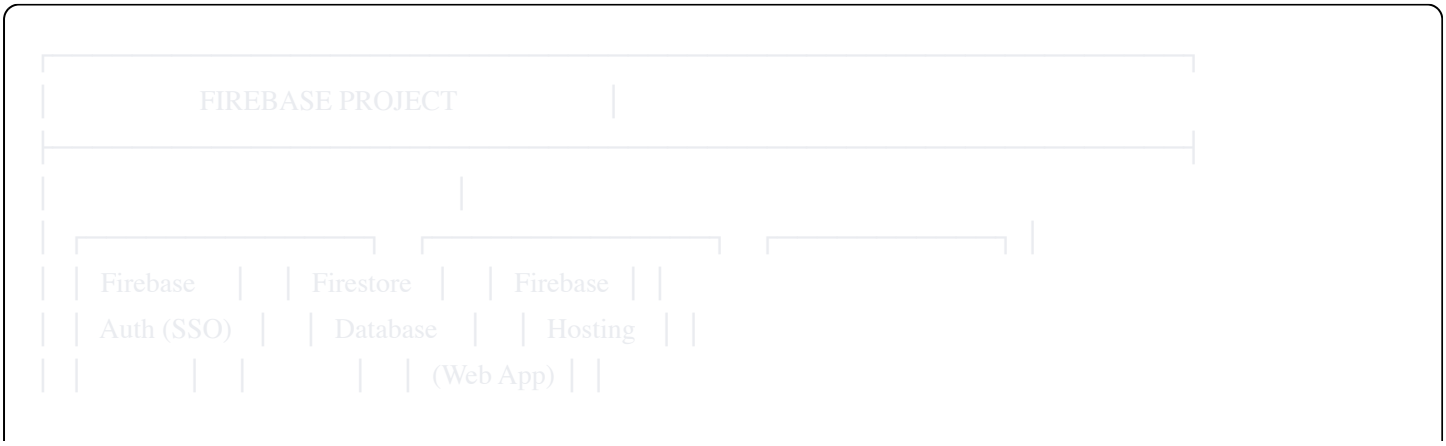
Consideration	Firebase	Raw GCP (Cloud Run + Python)	AWS Lambda + Python
Auth + Google SSO	Built-in, ~10 lines of config	Manual OAuth setup	Manual OAuth + Cognito
Database	Firestore included, real-time capable	Need to provision Cloud SQL or Firestore separately	DynamoDB (different paradigm)
Hosting (web app)	Firebase Hosting, zero config	Need Cloud Run or App Engine	S3 + CloudFront
Serverless functions	Cloud Functions, tight integration	Cloud Functions or Cloud Run	Lambda
Learning curve	Moderate — great docs, huge community	Steeper — more moving parts	Steeper + vendor lock-in away from Google ecosystem
Cost at 50 users	Free tier covers almost everything	Free tier covers most	Free tier covers most
Future flexibility	Can eject to raw GCP anytime	Already there	Different ecosystem entirely

Why TypeScript over Python:

- Firebase's tooling, docs, and SDKs are TypeScript-first.
- The Google APIs client libraries for Node.js are more mature for Tasks and Drive.
- TypeScript catches bugs at compile time — valuable when you're learning.
- The entire stack (frontend + backend) uses one language.

Verdict: Firebase + TypeScript gives us the fastest path to MVP with the least infrastructure management. If you hit a wall with TypeScript, individual Cloud Functions *can* be written in Python — Firebase supports both runtimes side by side.

System Components



```
│  └──────────┬──────────┘  └──────────┬──────────┘  └──────────┬──────────┘  │
│             │                        │                        │
│  ┌──────────┴────────────────────────┴────────────────────────┴──────────┐  │
│  │                  Cloud Functions (TypeScript)                          │
│  │                                                                        │
│  │  1. Drive Watcher — detects new transcripts                            │
│  │  2. Task Extractor — calls AI model                                    │
│  │  3. Notification Sender — emails proposed tasks                        │
│  │  4. Task Creator — writes to Google Tasks API                          │
│  │  5. API endpoints — serves the approval web app                        │
│  └────────────────────────────────────────────────────────────────────────┘  │
│                                    │
└──────────────────────────────────────────────────────────────────────────────┘

External Services:
  ├── Google Drive API (read transcripts)
  ├── Google Tasks API (create tasks)
  ├── Google Calendar API (identify meeting attendees)
  ├── AI Provider API (extract action items)
  └── Gmail API or Firebase Email (send notifications)
```

---

3. Detailed Component Breakdown

3.1 — Authentication & User Management

What it does: Users sign in with their Google Workspace account. The app stores their preferences and the OAuth tokens needed to act on their behalf (reading their Drive, creating their Tasks).

Key decisions:

- Firebase Auth with Google provider handles SSO.
- On first login, we trigger an OAuth consent flow requesting specific Google scopes.
- Tokens are stored encrypted in Firestore, per user.
- Users can toggle the service on/off from the web app.

Google Scopes needed (narrow):

Scope	Purpose
`drive.readonly`	Read meeting transcripts/notes from Drive
`tasks`	Create and manage tasks in Google Tasks
`calendar.events.readonly`	Identify meeting attendees and match to users
`gmail.send` *(or use Firebase email)*	Send notification emails

Firestore user document structure:

```
users/{uid}
  ├── email: string
  ├── displayName: string
  ├── isActive: boolean (on/off toggle)
  ├── tokens: { encrypted OAuth tokens }
  ├── preferences: {
  │     autoApprove: boolean (false by default)
  │     notifyVia: "email"
  │   }
  └── createdAt: timestamp
```

Workspace admin coordination required:

- Configure OAuth consent screen (internal app — simpler, no Google review needed).
- Whitelist the app's client ID.
- This is a one-time setup but can take time — start this conversation early.

3.2 — Transcript Detection (Drive Watcher)

What it does: Detects when a new meeting transcript appears in a user's Google Drive.

Approach — polling via scheduled function:

Google Drive push notifications (webhooks) are unreliable for transcripts because they require per-user channel setup and expire after ~1 week. For 50 users, a simpler approach:

- A Cloud Function runs every 5–10 minutes on a schedule.
- For each active user, it queries their Drive for new files matching transcript patterns (Google Meet transcripts have a predictable naming format and MIME type).
- New transcripts are queued for processing.
- We track already-processed file IDs in Firestore to avoid duplicates.

Why not Drive push notifications: They require refreshing watch channels per user, add complexity, and don't improve latency meaningfully for a post-meeting workflow. Polling every 5–10 min is fine since users won't expect instant processing the moment a meeting ends.

Firestore tracking:

```
processedTranscripts/{uniqueId}
  ├── driveFileId: string
  ├── userId: string (who triggered the detection)
  ├── meetingTitle: string
  ├── detectedAt: timestamp
  ├── status: "pending" | "processing" | "completed" | "failed"
  └── attendees: string[] (emails, from Calendar API)
```

3.3 — AI Task Extraction

What it does: Takes a transcript, identifies action items, assigns owners, and outputs structured data.

Model selection:

We'll build a provider-agnostic extraction layer so the AI model can be swapped. For MVP, recommend starting with **Claude (Anthropic API)** or **GPT-4o (OpenAI)** — both handle long transcripts well and produce reliable structured output.

The extraction function:

1. Receives the transcript text.
2. Calls the AI model with a carefully designed prompt.
3. Parses the response into structured task objects.
4. Returns them for review.

Prompt design principles (from the brief):

- Reflect uncertainty when transcript is unclear ("might be X vs Y").
- Never confidently invent specifics.
- Include enough context for the user to verify against the transcript.
- Keep task text "public-ish" — no sensitive details copied verbatim. Flag sensitive items as neutral pointers.

Output structure:

```typescript
```

```typescript
interface ExtractedTask {
  id: string;
  title: string;                // concise task title
  description: string;          // context from transcript
  assignee: string;             // email or name as mentioned
  confidence: "high" | "medium" | "low";
  transcriptExcerpt: string;    // relevant portion for verification
  isSensitive: boolean;         // if true, description is generic + points to source
  sourceFileId: string;         // Drive file ID for linking
  dueDate?: string;             // if mentioned in transcript
}
```

Cost considerations:

- Average transcript: ~5,000–15,000 tokens (30–60 min meeting).
- Long meeting (2+ hours): ~30,000–50,000 tokens.
- At 50 users averaging 3 meetings/day, expect ~150 API calls/day.
- Estimated monthly cost: $20–80 depending on model and meeting volume.

3.4 — Approval Flow (Email + Web App)

What it does: After extraction, each attendee gets their proposed tasks for review before anything is created.

Email notification:

- Simple email listing proposed tasks with approve/reject links.
- Each link points to the web app with a secure, time-limited token.
- Quick actions in email: "Approve All" link for when the user trusts the extraction.

Web app (Firebase Hosting + simple React/HTML):

The approval UI is minimal:

- List of proposed tasks from the latest meeting.
- For each task: title, description, confidence indicator, link to transcript.
- Actions per task: Approve ✓ | Edit ✏️ | Reject ✗
- Bulk actions: Approve All | Reject All.
- Settings page: on/off toggle, connected account status, preferences.

Approval states in Firestore:

```
proposals/{meetingId}/tasks/{taskId}
  ├── ...extractedTask fields
  ├── assigneeUid: string
  ├── status: "pending" | "approved" | "rejected" | "edited" | "created"
  ├── editedTitle?: string
  ├── editedDescription?: string
  ├── reviewedAt?: timestamp
  └── createdTaskId?: string (Google Tasks ID, after creation)
```

Expiration: Proposals expire after 48 hours. Unclaimed proposals are not created. This keeps things clean and avoids stale tasks appearing days later.

3.5 — Task Creation (Google Tasks API)

What it does: Takes approved tasks and creates them in the user's Google Tasks.

Key details:

- Each task includes: title, notes (with description + Drive link to transcript), due date if extracted.
- Tasks are created in a dedicated "TaskBot" task list (auto-created on first use) to keep them separate from the user's personal tasks.
- The Drive link in the task notes lets the user jump straight to the transcript for verification.
- No hard deletes — if a user changes their mind, they mark the task complete or delete it themselves in Google Tasks.

Google Tasks limitations (accepted for MVP):

- No shared task lists — each user gets their own copy of a task.
- No rich formatting in task notes.
- No sub-tasks natively tied to a parent "meeting" task (we work around this with the dedicated list).

---

4. Data Flow Summary

```
Meeting ends
    │
    ▼
Transcript appears in organizer's Drive (Google handles this)
    │
    ▼
[Drive Watcher] polls, detects new transcript
    │
    ▼
[Calendar API] identifies meeting attendees
```

```
    │
    ├── Filter: only users who are signed up + active
    │
    ▼
[Task Extractor] reads transcript, calls AI model
    │
    ├── Returns: structured task proposals per attendee
    │
    ▼
[Firestore] stores proposals with status "pending"
    │
    ▼
[Notification Sender] emails each attendee their proposals
    │
    ▼
User opens email → clicks through to web app
    │
    ├── Approves / edits / rejects each task
    │
    ▼
[Task Creator] creates approved tasks in user's Google Tasks
    │
    ├── Updates Firestore status to "created"
    │
    ▼
Done. Task includes link back to Drive transcript.
```

---

5. Work Packages & Estimates

Below are the major work packages in suggested build order. Estimates assume we're both working on this, with you learning TypeScript/Firebase as we go. Times include learning overhead.

#	Work Package	What's Included	Estimate
WP1	**Project Setup & Auth**	Firebase project, Firestore config, Firebase Auth with Google SSO, OAuth consent flow, token storage, basic user document	1–1.5 weeks
WP2	**Drive Watcher**	Scheduled Cloud Function, Drive API integration, transcript detection logic, deduplication, Firestore tracking	1 week
WP3	**AI Extraction Engine**	Provider-agnostic wrapper, prompt engineering, structured output parsing, confidence scoring, sensitivity detection	1–1.5 weeks
WP4	**Approval Web App**	Firebase Hosting setup, simple frontend (task list, approve/edit/reject), Firestore reads/writes, settings page	1.5–2 weeks
WP5	**Email Notifications**	Email template, secure approval links, "Approve All" shortcut, token generation/validation	0.5–1 week
WP6	**Task Creation**	Google Tasks API integration, dedicated list management, Drive link insertion, status tracking	0.5–1 week
WP7	**Integration Testing & Polish**	End-to-end testing with real meetings, error handling, edge cases, retry logic, logging	1–1.5 weeks
WP8	**Workspace Admin Coordination**	OAuth consent screen config, scope approval, deployment to production project	0.5 week (parallel)

Total estimate: 7–10 weeks

The range accounts for learning curve and the unpredictability of Google API quirks. WP8 should run in parallel starting from week 1.

---

6. Risk Register

Risk	Impact	Mitigation
Transcript quality is poor	AI extracts wrong or hallucinated tasks	Confidence scoring + mandatory human approval before creation. Drive links for verification.
Workspace admin delays	Can't deploy to THG environment	Build and test on your own Workspace. THG deployment is a configuration change, not a rebuild.
Google Drive polling misses transcripts	Tasks never get proposed	Logging + monitoring on the watcher. Manual "process this meeting" button in the web app as fallback.
OAuth token expiry	Tasks fail to create	Refresh token handling built into WP1. Alert user if re-auth is needed.

Risk	Impact	Mitigation
AI provider outage or rate limits	Processing delays	Provider-agnostic wrapper allows fallback. Queue + retry logic.
User doesn't check email / web app	Proposals expire unused	48h expiration is deliberate — avoids stale tasks. Can adjust window based on feedback.
Meeting attendees not matched to users	Tasks proposed for wrong people or missed	Match on email from Calendar API. Unmatched attendees are flagged for manual assignment in the web app.

---

7. What's Explicitly Out of Scope (MVP)

- Real-time in-meeting interaction (moved to post-MVP).
- Meeting mode selector (standup vs. long meeting).
- Zoom or non-Google Meet sources.
- External task systems (Copper, Asana, etc.).
- Shared/team task views.
- Mobile app.
- Org-wide admin dashboard.

---

8. Open Questions to Resolve Before Starting

1. **AI Provider:** Do you have a preference or existing account with OpenAI / Anthropic / Google (Gemini)? Or should we evaluate during WP3?
2. **Email sending:** Firebase has basic email extensions, but for styled emails we may want a service like SendGrid (free tier covers our volume). Preference?
3. **Transcript access:** Google Meet transcripts are stored in the *organizer's* Drive by default. For attendees to process them, the organizer needs to share or we need the organizer's token. Do meetings at THG typically have a consistent organizer, or does it vary? This affects how we architect the Drive Watcher.
4. **Domain:** Do we need a custom domain (e.g., taskbot.thehackinggames.com) or is a Firebase default URL fine for MVP?

---

9. Suggested Next Steps

1. **Align on this scope** — flag anything that feels wrong or missing.
2. **Resolve the open questions** above.

3. **Start WP8 immediately** — send the Workspace admin request to THG's IT.
4. **Begin WP1** — set up the Firebase project and auth flow together.